FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as "the unacknowledged use, as one's own work, of work of another person, whether or not such work has been published" (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

David Farrugia	
Student Name	Signature
Gianluca Amato	GAS.
Student Name	Signature
Student Name	Signature
Student Name	Signature

ICS5111	ICS5111-Project_GianlucaAmato_DavidFarrugia
Course Code	Title of work submitted

02/02/2026

Date

Retrieval-Augmented Generation (RAG) Pipeline in Business and Finance

Gianluca Amato (0344803L)
gianluca.amato.24@um.edu.mt
M.Sc. in Data Science
University of Malta
Msida, Malta

David Farrugia (0015004H)
david.farrugia.22@um.edu.mt
M.Sc. In Artificial Intelligence
University of Malta
Msida, Malta

Abstract

This report presents the design and evaluation of a Retrieval-Augmented Generation (RAG) pipeline for business and finance, combining unstructured financial text with structured macroeconomic indicators. The proposed system integrates two data sources from Kaggle and the World Bank Open Data repository before implementing and comparing three retrieval approaches: BM25, Dense Passage Retrieval (DPR), and Sentence-Transformers. This project also finds that Sentence-Transformers perform better than both BM25 and DPR, especially for diverse and statement-based queries. Highlighting the importance of using shared semantic representations when working with mixed types of data. The results also show that dense retrievers trained on question–answering data do not perform well outside their intended task, which suggests the need for more domain-specific training.

CCS Concepts

• **Information systems** → **Question answering**; **Top-k retrieval in databases**; **Similarity measures**.

Keywords

Retrieval-Augmented Generation, RAG, Mining Large-Scale Data, Natural Language, Information Retrieval, Augmented Generation, Sparse Techniques, Dense Techniques, Group Project

1 Introduction

Financial markets produce large volumes of varying information, including news articles, analyst commentary, company disclosures, and economic indicators [8]. Although such data is widely accessible nowadays, extracting readable and timely insights remains challenging, as market behaviour is often influenced by both qualitative factors, such as news reports, and quantitative signals, such as changes in economic indicators [3]. Manually synthesising these sources is very time-consuming and prone to bias, particularly in fast-moving market environments.

This project addresses the problem of automatically retrieving and summarising financial and market-related information to support high-level analytical queries. The proposed system aims to explain market movements, summarise company-related news, and contextualise financial conditions using economic indicators. To achieve this, a Retrieval-Augmented Generation (RAG) pipeline is developed that integrates unstructured financial text from a Kaggle dataset[1], with structured data from the World Bank Open Data repository[2]. By utilising generated outputs, the system seeks to produce concise, reliable, and interpretable summaries suitable for decision support in business and finance.

Therefore, a retrieval-augmented approach is particularly suitable for this task, as stand-alone generative models may produce fluent but factually incorrect outputs, which is an issue in finance-sensitive applications. By basing generation on retrieved evidence, the proposed RAG pipeline has a chance of improving factual reliability and transparency while allowing structured economic indicators and unstructured textual data to be leveraged [9].

1.1 Aims

The main objectives of this project are:

- To design and implement a retrieval-augmented generation (RAG) pipeline for financial and market-related information access.
- To retrieve relevant financial text and structured economic indicators in response to user queries.
- To implement and compare sparse and dense retrieval methods in terms of relevance and coverage.
- To generate relevant summaries that support high-level financial analysis.
- To critically evaluate the strengths and limitations of retrieval-augmented approaches in the financial domain.

2 Related Work

This section covers some of the related work for both sparse and dense retrieval techniques.

2.1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) was introduced as a combined framework by Lewis et al. [9], who proposed combining neural retrieval with generative language models to ensure that the generated text is supported by real, retrieved information rather than only the model's internal knowledge. Unlike purely parametric models, RAG dynamically retrieves relevant documents and conditions generation on this evidence, improving factuality and adaptability across domains as mentioned by Shuster et al. [14] and Zhand et al. [17]. In business and finance, this paradigm has gained popularity mostly due to the rapidly changing nature of financial information, where static model knowledge quickly becomes outdated and redundant.

[1] https://www.kaggle.com/sbhatti/financial-sentiment-analysis **[Last accessed on: February 3, 2026]**

[2] https://data.worldbank.org/ **[Last accessed on: February 3, 2026]**

Recent studies such as Chen et al. [1] have applied RAG-style architectures to tasks such as financial question answering and earnings report analysis, as well as market news and financial document summarisation, demonstrating improvements in factual consistency and interpretability compared to stand-alone generative models. Domain-specific RAG systems often rely on curated corpora such as financial news articles, company filings (e.g. SEC 10-K and 10-Q reports) as mentioned in Loughran et al. [10], and macroeconomic indicators, which are statistics that measure the economic health, performance, and future trends of a country or region, allowing models to ground responses in verifiable sources rather than relying on latent knowledge alone.

2.2 Sparse Retrieval

Mao Y. et al. [11] used RAG to answer open-domain questions. In their study, the authors made use of a Pre-trained Language Model (PLM) for the purpose of augmenting queries with heuristically discovered relevant context. This was done to simplify the retrieval of relevant passages by providing richer background information. Through this methodology, it was deduced that explicitly expressing the search intent resulted in improved results from the RAG system. Furthermore, the authors stated that the performance obtained from sparse retrieval was comparable or outperformed results from dense retrieval methods. Lastly, the usage of BM25 with augmented queries outperformed a similar usage of BM25 in the original queries [11].

Wang Y. et al. [15] conducted an evaluation of both dense and sparse RAG methodologies, including the sparse retrieval methods BM25 and TF-IDF. This evaluation was performed on 4 datasets, with the first 2 datasets containing a small amount of data and the last 2 datasets containing large amounts of data. Within each size category, one dataset contained clear and concise queries, whereas the other consisted of semantically similar queries with altered surface expressions. The results of this study showed that whilst the dense retrieval algorithm displayed greater stability in its performance, both sparse algorithms had a significant advantage regarding their resource efficiency, requiring only approximately 1/3 of the memory resources used by dense methods. It was also observed that BM25 consistently outperformed TF-IDF when it came to the datasets with altered queries [15]. This could be due to the normalisation BM25 performs on the length of the queries. However, the performance of the BM25 retrieval method decreased in performance when it was performed on the large altered dataset, while the dense method kept its stability [15]. This information suggests that sparse methods struggle on larger and more complex datasets.

2.3 Dense Retrieval

Dense retrieval is an alternative to sparse retrieval methods by representing both queries and documents as continuous vectors in a shared feature space. One influential framework in this domain is Dense Passage Retrieval (DPR), proposed by Karpukhin et al. [6], which introduces a dual-encoder architecture in which separate neural encoders are trained for questions and passages using contrastive learning. DPR demonstrated improvements over BM25 in open-domain question answering tasks by enabling semantic matching and not only on lexical overlap. This approach inspired a wave of research, including extensions such as ANCE (Xiong et al. [16]), which improves negative sampling strategies, and ColBERT (Khattab and Zaharia [7]).

More recently, general-purpose embedding models such as those provided by Sentence-Transformers (Reimers and Gurevych [13]) have enabled dense retrieval without task-specific training, making semantic search more accessible and scalable. In the context of RAG systems, Lewis et al. [9] demonstrated how dense retrievers combined with generative models tend to outperform models relying solely on their internal, pre-trained knowledge, without accessing external documents or databases. Furthermore, Izacard and Grave [5] further showed that multi-document dense retrieval improves both factual accuracy and robustness in generative QA systems. These studies help to motivate the use of dense retrieval in this project.

3 Design & Implementation

This section describes the architecture and workflow of the Retrieval-Augmented Generation (RAG) pipeline implemented for this project. The pipeline follows a modular design consisting of:

1. corpus construction and preprocessing
2. indexing and retrieval (sparse and dense)
3. answer generation using a sequence-to-sequence model
4. comparative evaluation using retrieval- and generation-level metrics

The implementation is provided as a single Jupyter notebook, with reproducibility supported via fixed random seeds and descriptive markdowns describing each section.

3.1 Environment Setup

The system is implemented in Python 3.11.9, which is recommended to avoid compatibility issues with deep learning and NLP libraries. Furthermore, this project can be accessed through GitHub[3], which contains the full source code and datasets utilised.

A dedicated virtual environment is used to isolate project dependencies, and all dependencies were installed using pip. All required Python packages are listed explicitly in a `requirements.txt` file and include libraries for natural language processing, neural retrieval, data handling, evaluation and visualisation. Key dependencies include `transformers`, `sentence-transformers`, `faiss-cpu`, `scikit-learn`, `nltk`, `pandas` and `torch`.

To ensure consistent behaviour across runs, a fixed random seed is set for Python's `random` module and NumPy. This guarantees consistent dataset processing, embedding generation, and retrieval results when executing the pipeline over and over.

Lastly, this project follows a structured directory layout that separates raw inputs, intermediate datasets and final outputs. Input datasets are expected to be placed in a predefined directory, and runtime validation checks ensure that the required files exist and conform to the expected formats. Temporary and output directories are created automatically if they are not present, preventing manual setup errors.

[3] https://github.com/DavidF-22/ICS5111-MiningLargeScaleData_Project [**Last accessed on: February 3, 2026**]

3.2 Overall Project Pipeline

The proposed system follows a RAG architecture that integrates financial data sources into a question-answering pipeline. The core section of this implementation can be seen in `Figure 1`

The pipeline begins with the construction of a unified corpus derived from two data sources: (i) an unstructured financial sentiment dataset obtained from Kaggle and (ii) structured macroeconomic indicators retrieved programmatically from the World Bank Open Data API, which includes country-level annual indicators such as GDP growth, inflation rate, unemployment rate, and government debt.

Following corpus construction, two parallel strategies are applied. First, a sparse representation is created by applying token-based text normalisation, including lowercasing, stopword removal, and optional retention of four-digit year tokens. This representation is used for term-based retrieval with a BM25-style scoring function. Second, dense representations are generated using neural embedding models, enabling semantic similarity search beyond exact keyword matching. Two dense retrieval approaches are implemented: Sentence-Transformers using a shared feature space for queries and documents, and Dense Passage Retrieval (DPR), which employs separate encoders for questions and contexts and indexes document embeddings utilising FAISS for efficient similarity search.

During inference, a query is processed by the sparse and dense retrievers sequentially. Each retriever returns the top-k most relevant documents from the corpus. The texts are concatenated to form a contextual passage. This context, together with the original question, is injected into a structured prompt and passed to a pretrained sequence-to-sequence generative model (FLAN-T5[4]). The model produces a single-sentence answer that is explicitly grounded in the retrieved evidence.

Finally, the pipeline supports comparative evaluation across retrieval strategies. Retrieval quality is assessed using ranking-based metrics such as Precision@K, Recall@K, F1@K, and Mean Reciprocal Rank (MRR), while generation quality is evaluated using BLEU and ROUGE scores. This modular design allows for systematic analysis of how different retrieval methods affect answer quality.

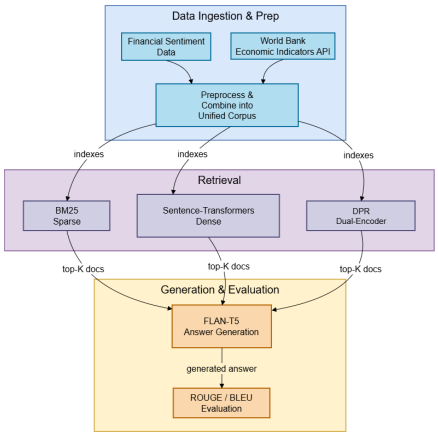

Figure 1: Project System Diagram

[4]https://huggingface.co/google/flan-t5-large [**Last accessed on: February 3, 2026**]

3.3 Corpus Construction & Preprocessing

The corpus was formed through the integration of two data sources: financial sentiment text and structured macroeconomic indicators. This integration allows retrieval models to access both qualitative and quantitative information within a single searchable area. Since retrieval systems operate primarily on textual input, structured economic records were transformed into natural-language (NL) sentences while preserving relevant metadata such as year values. Each record in the corpus, therefore, maintains both its original text and a processed representation. This allowed for compatibility across different retrieval strategies while preserving the traceability of the data sources.

To ensure consistency and reduce noise across the corpus, a shared preprocessing pipeline was applied to all textual data before indexing. The pipeline standardises text by converting it to lowercase, tokenising sentences into individual terms and removing common English stopwords that contribute little semantic value in retrieval tasks. Non-alphabetic tokens are also removed to eliminate punctuation and formatting artefacts, as they can increase vocabulary size. Both raw and processed text are stored so that retrieval methods can operate on the most suitable representation when required.

Structured economic data retrieved from the World Bank Open Data API were subsequently converted into descriptive sentences, for example: `"In 2024, Malta's GDP growth (%) was 6.79."` so that they could be indexed alongside financial text using the same retrieval mechanism. After preprocessing, both datasets were aligned to a shared schema to ensure consistent metadata fields before integration. Missing metadata fields were added where necessary to maintain structural compatibility across sources. The datasets were then merged into a single unified corpus, allowing for the comparative analysis of both sparse and dense retrieval techniques within a common search space. Before saving the dataset, duplicate entries were identified, and a total of 520 duplicate rows were removed to prevent bias in the results, as these rows could have caused certain samples to appear multiple times, possibly affecting the results.

Lastly, it is worth noting that a brief exploratory data analysis (EDA) section was done after the final dataset was created to verify dataset consistency and basic characteristics. However, since this analysis was not part of the original scope of the project, details are provided separately in Appendix B.

3.4 Sparse Retrieval

BM25 was selected as the sparse retrieval method as it accounts for document length normalisation and applies a logarithmic saturation function when computing term weights. This avoids the method placing priority on frequently occurring words [12].

To apply the BM25 ranking function, a relevance score was computed for every document in the corpus. For each query term, the term frequency (TF) and inverse document frequency (IDF) were calculated. The results were then combined using the BM25 scoring formula, and the resulting values from all query terms were summed to obtain the final BM25 score for each document. The TF was calculated through the following formula [4]:

$$\mathrm{TF}(t,d) = \frac{\mathrm{freq}(t,d)}{\mathrm{freq}(t,d) + k_1\left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)}$$

where:

- t is a term,
- d is a document,
- $|d|$ is the length of document d,
- avgdl is the average document length in the corpus,
- k_1 controls term frequency scaling.
- b controls document length normalisation.

The values selected for k_1 and b were 1.2 and 0.75 respectively as those values are known to work well for most corpus' [2]. The terms were gathered by processing and tokenising the inserted query.

The IDF is calculated with the following formula [4]:

$$\mathrm{IDF}(t) = \log\left(\frac{N - n_t + 0.5}{n_t + 0.5}\right)$$

where N is the total number of documents within the corpus and n is the number of documents containing the term t.

Lastly, the overall BM25 score is then calculated by using the TF and IDF in the following formula [4]:

$$\mathrm{Score}(q,d) = \sum_{t \in q} \mathrm{IDF}(t)\,\mathrm{TF}(t,d)$$

The above formulas were computed for every term found within the tokenised query. Afterwards, the documents with the highest BM25 scores are displayed and used as a prompt to generate a response for the query.

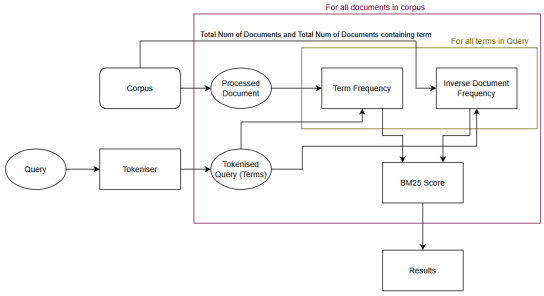

Figure 2: BM25 retrieval process flowchart

3.5 Dense Retrieval

Two dense retrieval techniques were implemented and compared in this project: Sentence-Transformers and Dense Passage Retrieval (DPR). Both techniques operate by encoding documents into dense vector representations, after which they are evaluated and compared with the sparse retrieval techniques.

3.5.1 Sentence-Transformers. The first dense retrieval technique uses a Sentence-Transformers model to map both documents and queries into the same semantic feature space. Before encoding, the system verifies that each document contains a valid text field, ensuring that the corpus can be processed consistently.

All document texts are extracted and encoded using a lightweight transformer model from HuggingFace[5] (all-MiniLM-L6-v2) designed for semantic similarity tasks. The outputs are normalised so that similarity comparisons depend only on directional alignment in the vector space rather than the differences in length.

Furthermore, the user query is encoded using the same model and normalisation process. Cosine similarity is then computed between the query embedding and all document embeddings. Documents are ranked according to similarity scores, and the top-k passages are returned together with metadata and ranking information. This approach enables semantic retrieval even when queries and documents share few or no keywords. All of these stages can be seen in `Figure 3`

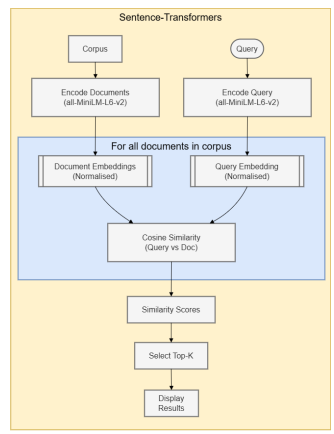

Figure 3: Sentence-Transformers retrieval workflow

3.5.2 Dense Passage Retrieval (DPR). The second dense retrieval technique is based on Dense Passage Retrieval, which makes use of a dual-encoder architecture consisting of two independent neural encoders: one specialised for queries and the other for documents. This separation allows each encoder to learn representations optimised for a specific role.

During preprocessing, all documents are encoded using the context encoder[6]. To prevent memory errors and improve efficiency, documents are processed in batches. Each batch is tokenised and passed through the encoder, and the resulting embeddings are stored and concatenated into a single matrix forming the dense index.

To enable scalable retrieval, document embeddings are indexed using C, which provides efficient similarity search over large embedding collections. An inner-product similarity index is used, which is compatible with DPR embeddings and allows fast retrieval without scanning the entire corpus at query time.

During inference, the user query is tokenised and encoded using the question encoder[7]. The resulting query embedding is searched

[5] https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 **[Last accessed on: February 3, 2026]**
[6] https://huggingface.co/facebook/dpr-ctx_encoder-single-nq-base **[Last accessed on: February 3, 2026]**
[7] https://huggingface.co/facebook/dpr-question_encoder-single-nq-base **[Last accessed on: February 3, 2026]**

against the FAISS index to retrieve the top-k most similar document vectors. The relevant documents are then returned and later supplied to the generative model to produce grounded responses, similarly to the techniques above. All of these stages can be seen in `Figure 4`

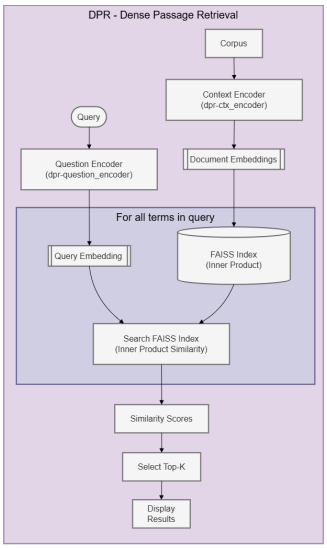

Figure 4: DPR retrieval workflow

4 Evaluation & Comparative Analysis

This section presents a comparative evaluation of the RAG pipeline. Both retrieval-level and generative-level evaluations are conducted to highlight the strengths and limitations of each approach in the context of financial and economic information retrieval.

4.1 Retrieval-level Evaluation

To evaluate the performance of the RAG systems, each system was run using a set of seven distinct queries. These queries are listed from a to g. The query labelled (b) was chosen to be an augmented version of the query labelled (c), having more detail included into the question. This was done to assess the effect that the nature of the query has over the results. The relevance of the documents retrieved by each RAG system was then determined through human annotation.

Label	Query
(a)	How did inflation change in Italy after 2020?
(b)	Investors are worried about rising inflation amid political uncertainty
(c)	Investors are worried about inflation in politics
(d)	What happened to unemployment in Malta after COVID?
(e)	How did GDP change in European countries after the financial crisis?
(f)	What trends are observed in European interest rates in recent years?
(g)	How has economic growth evolved in the Eurozone after 2015?

Table 1: List of evaluation queries

After the results were annotated in terms of relevancy, the Precision@k and MRR scores were calculated for every list of results. The Precision@k score was calculated using the formula $\frac{\text{Number of Relevant Items in Top } K}{K}$, where K is the total number of documents returned per query. The MMR Score was calculated with the formula $\frac{1}{J}$, where J is the ranking of the first relevant document. These results show that Sentence-Transformers was the highest performing retriever, whilst DPR was the worst performing retriever.

Sentence-Transformers achieved the highest retrieval performance largely because it encodes both queries and documents into a shared semantic feature space. This allowed it to capture similarities other than just keyword overlap, which made it effective for the varied and queries used in the evaluation. This alignment makes it more robust to the varied phrasing, financial terminology, and structural diversity present in our corpus. In contrast, dense retrievers such as DPR, which use separate encoders trained predominantly on question and answer datasets, may underperform when the task changes from their training distribution. The reliance on QA-oriented training means DPR's representations are ideal for answer span matching (where a start and end point are clearly stated), which likely contributed to its lower effectiveness on our statement-based and domain-specific queries.

These results show that getting better retrieval results in mixed-domain text requires more than just using standard models. Models like Sentence-Transformers can improve further if they are fine-tuned on financial data or trained with more contrastive examples. Adding simple data enhancements, such as paraphrasing or expanding queries with domain terms, can also help the model learn finer semantic differences.

As for BM25, the retriever performed poorly on queries that did not involve matching explicit terms within the corpus. In particular, it struggled with queries requiring temporal reasoning (identifying documents published after a specific date) or grouped terms, such as retrieving countries that belong to a broader category or grouping (member states of the European Union). Additionally, BM25 exhibited the slowest retrieval time among all evaluated retrievers. This may be attributed to the need to compute BM25 relevance scores for each document in the corpus, requiring multiple calculations per document before ranking and selecting the most relevant results.

In regards to the augmented query, it can be seen that there was not much difference in the quality of the results in the BM25 and ST retrievers. The exception being the DPR Retriever, which had both lower Precision@k and MMR scores. Additionally, it can be noted that query f had the worst performance from all three retrievers, with none of the retrievers gathering any relevant documents. This could be due to the ambiguity of the query, having no specific dates or countries mentioned.

4.2 Generative-level Evaluation

The Generative-level evaluation was performed using Rouge1-F1 Score and SacreBLEU. The Rouge1-F1 score were consistently low across all retrievers, indicating that there was not much similarity between the answers created by the retrievers and the ideal answers. This could either be due to

1. The results gathered by the retrievers, as they could have gathered documents that are unrelated to the topic

2. The quality of the generated response from the model, as some responses were observed to either not have been generated or were generated as a single word.
3. The corpus itself, as it may not have contained the correct information to satisfy the requirements of the evaluation.

The SacreBLEU scores remained low overall, indicating that the generated texts were all of low quality. The Sentence-Transformers retriever achieved the lowest SacreBLEU score, while BM25 and DPR obtained identical scores of 1.49. This suggests that BM25 and DPR produced generated answers with greater similarity to the reference texts, whereas the outputs of the Sentence-Transformers retriever diverged more in retrieved content.

Metric	BM25	ST	DPR
SacreBLEU	1.49	0.65	1.49

Table 2: SacreBLEU scores across different retrievers

Retriever	Query	Precision@5	MRR	Rouge1 F1
BM25	(a)	0.6	1	0.47
BM25	(b)	0.2	0.5	0
BM25	(c)	0.2	0.5	0
BM25	(d)	0	0	0.23
BM25	(e)	0.4	0.5	0.12
BM25	(f)	0	0	0.13
BM25	(g)	0.8	1	0.33
ST	(a)	1	1	0.35
ST	(b)	0.4	1	0.13
ST	(c)	0.4	1	0.13
ST	(d)	0.6	1	0.23
ST	(e)	0.8	1	0
ST	(f)	0	0	0
ST	(g)	1	1	0.11
DPR	(a)	0.6	0.5	0.35
DPR	(b)	0	0	0.1
DPR	(c)	0.2	0.33	0.13
DPR	(d)	0	0	0
DPR	(e)	0	0	0.21
DPR	(f)	0	0	0
DPR	(g)	0.4	0.5	0.25

Table 3: Precision@k and MRR results per retriever and query

5 Conclusion & Future Work

Overall, a RAG pipeline for financial and market-related information was successfully created and implemented with three different retrievers, BM25, DPR and Sentence-Transformers. The Sentence-Transformers retriever was the highest performing retriever. This was attributed to its ability to encode both queries and documents into a shared semantic feature space, which gave it the capability to capture relationships beyond simple keyword overlap, improving its effectiveness when handling diverse evaluation queries.

The BM25 retriever ranked second among the three evaluated retrievers. The retriever was effective for queries involving direct term matching. However, its performance suffered on queries that required reasoning beyond surface-level term matching, such as those involving time-based ranges or identifying terms that belong to a grouping of words, such as countries that reside within continents. However, this limitation could be mitigated through the use of passage augmentation, as discussed in the related works, as enhancing documents with additional contextual information enables retrievers such as BM25 to better match queries that require reasoning or broader understanding. Furthermore, BM25 had the slowest retrieval time, likely due to the need to compute relevance scores across the entire corpus. As a result, BM25 becomes increasingly inefficient as corpus size grows, as processing time increases substantially with corpus size.

Lastly, DPR had the worst performance out of the three retrievers. As its encoders were trained on QA datasets, it had difficulties matching queries to relevant documents, resulting in lower retrieval performance. This suggests that there is a reduction in effectiveness when DPR is not used for its intended purpose.

For future works, the retrievers used could be improved within their efficiency, placing a larger focus on hyperparameter tuning and more optimised retriever structure. Additionally, a larger corpus could be used, as the current corpus size may have contributed to the low performance observed across most retrievers. Lastly, more query augmentation could be performed to help reduce ambiguity and improve retrieval performance for poorly defined queries, particularly for retrievers that rely on direct matches between query terms and documents.

Distribution of Work

The project work was divided among the group members. David Farrugia was primarily responsible for the early, data-centric sections of the report, abstract, introduction, dense retrieval background, corpus construction and preprocessing, dense retrieval implementation, and exploratory data analysis, focusing mainly on data handling, dense retrieval methods.

Gianluca Amato took primary responsibility for the sparse retrieval aspects of the project, as well as the evaluation and analytical components. This included the related work on sparse retrieval, the implementation of sparse retrieval, comparative evaluation, and the conclusion and future work sections.

Several sections, however, were completed collaboratively. These included the overall literature review on Retrieval-Augmented Generation (RAG), environment setup, overall system pipeline design, project notebook setup, data processing, the comparative evaluation experiments and the project presentation. This allowed for a more cohesive and complete final submission.

References

[1] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2022. FinQA: A Dataset of Numerical Reasoning over Financial Data. arXiv:2109.00122 [cs.CL] https://arxiv.org/abs/2109.00122

[2] Elastic. 2018. *Practical BM25 — Part 3: Considerations for Picking b and k1 in Elasticsearch.* https://www.elastic.co/blog/practical-bm25-part-3-considerations-for-picking-b-and-k1-in-elasticsearch Accessed: 2026-02-01.

[3] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs.CL] https://arxiv.org/abs/2312.10997

[4] GeeksforGeeks. 2025. What is BM25 (Best Matching 25) Algorithm? https://www.geeksforgeeks.org/nlp/what-is-bm25-best-matching-25-algorithm/ Accessed: 2026-02-02.

[5] Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty (Eds.). Association for Computational Linguistics, Online, 874–880. doi:10.18653/v1/2021.eacl-main.74

[6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. arXiv:2004.04906 [cs.CL] https://arxiv.org/abs/2004.04906

[7] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. arXiv:2004.12832 [cs.IR] https://arxiv.org/abs/2004.12832

[8] Michael Klesel and H. Felix Wittmann. 2025. Retrieval-Augmented Generation (RAG). *Business & Information Systems Engineering* 67, 4 (01 Aug 2025), 551–561. doi:10.1007/s12599-025-00945-3

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401 [cs.CL] https://arxiv.org/abs/2005.11401

[10] Tim Loughran and Bill McDonald. 2011. When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks. *The Journal of Finance* 66, 1 (2011), 35–65. doi:10.1111/j.1540-6261.2010.01625.x

[11] Yuning Mao, Pengcheng He, Xiaodong Liu, Yelong Shen, Jianfeng Gao, Jiawei Han, and Weizhu Chen. 2021. Generation-Augmented Retrieval for Open-Domain Question Answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 4089–4100. doi:10.18653/v1/2021.acl-long.316

[12] Aina Mardiah, Salsa Dillah, Dewi Fatmarani Surianto, Nur Fadilah, and Satria Gunawan Zain. 2025. Classification of Livin' by Mandiri Customer Satisfaction Using MLP with BM25 and TF-IDF Feature Weighting. *Kinetik: Game Technology, Information System, Computer Network, Computing, Electronics, and Control* 10, 3 (Aug. 2025), 355–366. doi:10.22219/kinetik.v10i3.2248

[13] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 3982–3992. https://aclanthology.org/D19-1410/

[14] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval Augmentation Reduces Hallucination in Conversation. arXiv:2104.07567 [cs.CL] https://arxiv.org/abs/2104.07567

[15] Yang Wang, Guanlin Dai, Song Ke, and Chao Zheng. 2025. Evaluating Sparse and Dense Retrieval in Retrieval-Augmented Generation Systems: A Study. In *Proceedings of the 2024 10th International Conference on Communication and Information Processing (ICCIP '24)*. Association for Computing Machinery, New York, NY, USA, 548–554. doi:10.1145/3708657.3708747

[16] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. arXiv:2007.00808 [cs.IR] https://arxiv.org/abs/2007.00808

[17] Wan Zhang and Jing Zhang. 2025. Hallucination Mitigation for Retrieval-Augmented Large Language Models: A Review. *Mathematics* 13, 5 (2025). doi:10.3390/math13050856

A Use of Generative AI

Generative AI (GenAI) tools were primarily used to support research, report development, and code debugging throughout the project. The tools utilised in this project are ChatGPT, Gemini, Claude and NotebookLM.

Most of the use of GenAI occurred during the background research section of the report. These tools were used to identify relevant academic topics, suggest potential references, and summarise main points from research papers. However, all suggested references were manually validated by consulting the sources to ensure accuracy, relevance and academic suitability before being cited in the report.

Additionally, once relevant sources were identified, the corresponding PDFs were provided to the tools, particularly NotebookLM, which allow us to discuss fundamental concepts and methodologies. This assisted in improving our understanding and ensuring that explanations included in the report remained accurate and aligned with the scope of the project.

These tools were also used to review draft sections of the report to assess whether they were in scope before integration into the main document. This included checking clarity, structure and alignment with the project objectives.

Lastly, GenAI primarily Gemini and Claude were utilised during code debugging, primarily to identify and deal with errors, suggest refactoring strategies, or clarify framework-specific behaviour. All suggested solutions were manually tested before being applied.

In summary, GenAI tools were used in this project as assistive tools to help us in: understanding core concepts, validation and documentation refinement and not as decision makers.

A.1 Prompts & Responses

A.1.1 Gianluca Amato. **Prompt 1:** *I am working on a Python RAG Pipeline using sparse methodologies. Can you help me create a flow so that I know where to start from, and how to proceed?*
Summarised Response: A pipeline flow was generated by ChatGPT, with explanations and possible tools for each stage within the flow. However, many of the possible tools described were high-level libraries, which were not used to adhere to the restrictions of the assignment. A suggested repo structure was also generated, which was not used.

Prompt 2: *How do you use Hugging face to generate responses, and what Hugging Face transformers do you recommend for generating a response using documents for a query?*
Summarised Response: ChatGPT explained methods for generating responses, giving code snippets. The code snippets had errors and needed to be readjusted manually. Afterwards, google/flan-t5-bas and bert-base-uncased were suggested as potential models to be used. However, the suggested models were not generating usable responses and were unused.

Prompt 3: *Can you rephrase this sentence so that it sounds more appropriate within an academic setting?*
Summarised Response: The query was reworded with numerous options, ranging from more formal phrasing to more academically

focused wording.

Prompt 4: *Can you convert this mathematic formula into LaTeX?*
Summarised Response: A LaTeX formula was generated to be used within the report. The formula was verified to be accurate.

Prompt 5: *My code should be generating a list of documents, alongside a BM25 score. However, the code is only generating BM25 scores of 0. What could be causing the issue?*
Summarised Response: A list of 5 possible fixes were recommended, 4 of which were wrong. The final fix was the correct one, which was that the dataset was being iterated over incorrectly, resulting in the column names being iterated over instead of the rows.

A.1.2 David Farrugia. **Prompt 1:** *"Can you give me a list of 10 references that are relevant to the project Retrieval-Augmented Generation (RAG) Pipeline in Business and Finance please?"*
Summarised Response: Ten potentially relevant references were suggested, each with a brief description about its focus and applicability. These references provided multiple starting points for exploring RAG architectures, dense retrieval techniques which were very helpful, and their application within business and financial domains. Similar prompts were used during the early research stage to identify how similar approaches had previously been applied, including the use of the World Open Data API.

Prompt 2: *"Based on the project description and objectives, is the following draft section in scope for a RAG pipeline in business and finance? (project description and a draft file where attached)"*
Summarised Response: The GenAI tool provided a draft section and identified areas that were well aligned with the project scope, and areas that needed some improvement. The tools also provided different variations of how the draft can be improved. For example, to include and expand more on the financial applications of RAG in the related work section.

Prompt 3: *"I am encountering this issue with this code snippet in the dense retrieval implementation. Can you help identify possible causes of the error and suggest potential fixes?"*
Summarised Response: Claude suggested multiple possible causes for the observed behaviour, including improper handling of embeddings and configuration issues within the retrieval snippet. It also proposed general debugging strategies like adding print statement through out the code, showing the output at each stage, which proved quite useful.

Prompt 4: *"When it comes to evaluating Sparse and Dense Retrieval techniques, what does the @k refer to in the metrics: precision@k, recall@k, etc...?"*
Summarised Response: It was then made clear that the parameter k represents the number of top-ranked retrieved items considered when computing the metrics. It clarified how precision@k measures the proportion of relevant items within the top k results, while recall@k evaluates how many of the total relevant items are retrieved within those top k positions. This explanation allowed me to better understand how performance is assessed and later how it's implemented in Python.

A.2 Individual Reflections

Each group member provides an individual reflection in this section, outlining their personal experience with the use of GenAI throughout the project.

A.2.1 Gianluca Amato. I have found Generative AI helpful for saving time and for covering my weaknesses when writing reports and code. Firstly, I initially struggled with starting the assignment due to the scope. I spent the majority of the first day scrolling through the assignment brief and trying to formulate a starting point. Eventually, I used ChatGPT to create a flow for me so that I could get an idea about where I should start from and what I should prioritise.

Secondly, ChatGPT assisted the report writing process. This was due to the large amount of information that needed to be summarised at times. Draft text was provided to the model to indicate the intended structure, and the content was then rewritten to be more concise and clearer. However, the AI sometimes used unclear or overly uncommon wording, which made the writing feel unnatural and disrupted the overall flow. GenAI was also used to reformat formulas into LaTeX for use within the report.

Thirdly, GenAI assisted in finding bugs that were hard to find, such as the wrong row/column of a dataframe being used. However, GenAI would, at times, hallucinate code and bugs incorrectly. If this advice was followed to the letter, the code would have been formatted into an unusable state. Thus, the recommended fixes were thoroughly inspected to make sure that they would not have a negative impact on the code.

Overall, GenAI is helpful only if the user knows how to properly handle it. In my case, I treat everything generated by GenAI as a template or a draft instead of as a finished result. I verify everything that is generated and I make sure to check for inconsistencies, possible improvements and any unnecessary/redundant sections. This approach has allowed me to utilise AI solely as a tool to save time and avoid including any AI hallucinated content into either the code or the report.

A.2.2 David Farrugia. As a student who strongly prefers hands-on and practical work, I often find theory-heavy components of projects, such as report writing and research, less engaging. This often resulted in these tasks being postponed until the last minute. However, the use of GenAI tools significantly changed my approach to these sections in the project.

GenAI tools helped make the theoretical sections more approachable and less overwhelming, particularly during the early stages of background research and finding references for the project. Tasks such as summarising papers and articles, clarifying definitions and foundational concepts and refining the structure and flow of the report became more manageable. As a result, these tools made the work easier and encouraged me to engage with the material instead of putting it off.

Using GenAI also highlighted how productivity can be improved when appropriate tools are used effectively. By reducing time spent

on repetitive tasks, more focus could be allocated to core project activities such as implementation and experimentation.

However, a conscious and responsible approach was adopted throughout the project. All GenAI outputs were treated strictly as guidance. Suggested material, like references, was manually reviewed and skimmed through to ensure validity before being integrated into the final work. This was particularly important to mitigate risks such as hallucinated information, oversimplified explanations, or inaccuracies.

B Exploratory Data Analysis (EDA)

A simple EDA section in the notebook was done to quickly understand the structure of the final dataset before applying the retrieval techniques. Using Plotly, interactive plots were generated to visualise the distribution of records across data sources using a histogram, and the number of missing values per column using a bar chart. Plotly was chosen for plotting because of its modern and clear plots compared to matplotlib.

The data source plot (`Figure 5`) shows that most entries originate from the Kaggle dataset, with fewer records from the World Bank source, indicating a noticeable source imbalance. Meanwhile, the missing values plot (`Figure 6`) shows that several columns contain missing data, especially metadata-related fields, which is expected since not all sources provide complete information. Overall, this step simply confirms dataset composition and basic data quality before continuing with the main workflow.

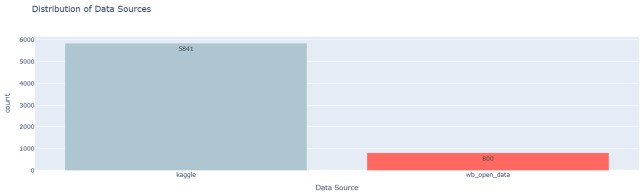

Figure 5: Distribution of Data Sources

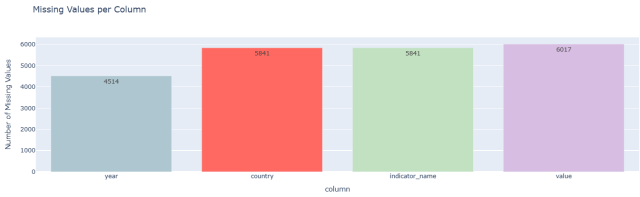

Figure 6: Missing Values per Column

C Comparative Analysis Tables

Rank	BM25	ST	DPR
1	Relevant	Relevant	Not Relevant
2	Relevant	Relevant	Relevant
3	Relevant	Relevant	Not Relevant
4	Not Relevant	Relevant	Relevant
5	Not Relevant	Relevant	Relevant

Table 4: Relevance judgements for Query (a)

Rank	BM25	ST	DPR
1	Not Relevant	Relevant	Not Relevant
2	Relevant	Relevant	Not Relevant
3	Not Relevant	Not Relevant	Not Relevant
4	Not Relevant	Not Relevant	Not Relevant
5	Not Relevant	Not Relevant	Not Relevant

Table 5: Relevance judgements for Query (b)

Rank	BM25	ST	DPR
1	Not Relevant	Relevant	Not Relevant
2	Relevant	Relevant	Not Relevant
3	Not Relevant	Not Relevant	Relevant
4	Not Relevant	Not Relevant	Not Relevant
5	Not Relevant	Not Relevant	Not Relevant

Table 6: Relevance judgements for Query (c)

Rank	BM25	ST	DPR
1	Not Relevant	Relevant	Not Relevant
2	Not Relevant	Relevant	Not Relevant
3	Not Relevant	Relevant	Not Relevant
4	Not Relevant	Not Relevant	Not Relevant
5	Not Relevant	Not Relevant	Not Relevant

Table 7: Relevance judgements for Query (d)

Rank	BM25	ST	DPR
1	Not Relevant	Relevant	Not Relevant
2	Relevant	Not Relevant	Not Relevant
3	Not Relevant	Relevant	Not Relevant
4	Relevant	Relevant	Not Relevant
5	Not Relevant	Relevant	Not Relevant

Table 8: Relevance judgements for Query (e)

Rank	BM25	ST	DPR
1	Not Relevant	Not Relevant	Not Relevant
2	Not Relevant	Not Relevant	Not Relevant
3	Not Relevant	Not Relevant	Not Relevant
4	Not Relevant	Not Relevant	Not Relevant
5	Not Relevant	Not Relevant	Not Relevant

Table 9: Relevance judgements for Query (f)

Rank	BM25	ST	DPR
1	Relevant	Relevant	Not Relevant
2	Relevant	Relevant	Relevant
3	Not Relevant	Relevant	Relevant
4	Relevant	Relevant	Not Relevant
5	Relevant	Relevant	Not Relevant

Table 10: Relevance judgements for Query (g)